

MODULE 1 (CONT.): MACHINE INSTRUCTIONS & PROGRAMS

Computers are built using logic circuits that operate on information in the form of binary digit whose values are 0 and 1. The most natural way to represent a number in a computer system is by a string of bits, called a binary number. A text character can also be represented by a string of bits called a character code.

Numeric Data Representation

	Decimal	Binary	Octal	Hexadecimal
Fixed Point	00	0000	00	0
	01	0001	01	1
	02	0010	02	2
	03	0011	03	3
	04	0100	04	4
	05	0101	05	5
	06	0110	06	6
	07	0111	07	7
	08	1000	10	8
	09	1001	11	9
	10	1010	12	A
	11	1011	13	B
	12	1100	14	C
	13	1101	15	D
	14	1110	16	E
	15	1111	17	F

Representation:

It's the representation for integers only where the decimal point is always fixed. i.e at the end of rightmost point. it can be again represented in two ways.

1. Sign and Magnitude Representation:

In this system, the most significant (leftmost) bit in the word as a sign bit. If the sign bit is 0, the number is positive; if the sign bit is 1, the number is negative.

The simplest form of representing sign bit is the sign magnitude representation.

One of the draw back for sign magnitude number is addition and subtraction need to consider both sign of the numbers and their relative magnitude.

Another drawback is there are two representation for 0(Zero) i.e +0 and -0.

2. One's Complement (1's) Representation:

In this representation negative values are obtained by complementing each bit of the corresponding positive number.

COMPUTER ORGANIZATION

For example 1s complement of 0101 is 1010. The process of forming the 1s complement of a given number is equivalent to subtracting that number from $2^n - 1$ i.e from 1111 for 4 bit number.

Two's Complement (2's) Representation Forming the 2's complement of a number is done by subtracting that number from 2^n . So 2's complement of a number is obtained by adding 1 to 1s complement of that number.

Ex: 2's complement of 0101 is $1010 + 1 = 1011$

NOTE: In all systems, the leftmost bit is 0 for positive number and 1 for negative number.



Signed integer represented in three ways.

Represent +12 and -12 in the following ways.

1. Sign and Magnitude Representation:

Sign bit	True Magnitude (actual binary bits)
----------	-------------------------------------

Ex: a. +12 is 01100 [4 bits are for representing number 12 and 1 bit is for sign. Hence]
b. -12 is 11100 [it is signed 5 bit number representation]

2. One's Complement (1's) Representation:

Sign bit	1's complement of actual binary bits
----------	--------------------------------------

EX: a. +12 is 01100 and b. -12 is 10011

3. Two's Complement (2's) Representation :

Sign bit	2's complement of actual binary bits
----------	--------------------------------------

Ex: a. +12 is 01100 and b. -12 is 10100

Note: unsigned (+ve) number representation remains in same manner in all three ways.

Department of Electronics & Communication

www.cambridge.edu.in



Addition and Subtraction – 2's Complement

Any subtraction operation is performed by means of 2's complement addition.
Hence the operation is addition and subtraction

Perform following operation in 2's complement system in signed 4 bit representation.

1. $(4) + (3)$ 2. $(-4) - (3)$ 3. $(4) + (-3)$ 4. $(-4) + (3)$

Solution: 1. $(4) + (3)$

$$\begin{array}{r} 4 \quad 0100 \\ + 3 \quad 0011 \\ \hline 7 \quad 0111 \end{array}$$

2. $(-4) + (-3)$

$$\begin{array}{r} -4 \quad 1100 \\ + (-3) \quad 1101 \\ \hline -7 \quad 11001 \end{array}$$

3. $(4) + (-3)$

$$\begin{array}{r} 4 \quad 0100 \\ - 3 \quad 1101 \\ \hline 1 \quad 10001 \end{array}$$

4. $(-4) + (3)$

$$\begin{array}{r} -4 \quad 1100 \\ + 3 \quad 0011 \\ \hline -1 \quad 1111 \end{array}$$

$$\begin{array}{r} +4 \quad 0100 \\ +3 \quad 0011 \\ \hline -4 \quad 1011 \\ +1 \\ \hline 1100 \\ -3 \quad 1100 \\ +1 \\ \hline 1101 \end{array}$$

$$\begin{array}{r} 0110 \\ +1 \\ \hline 0111 \\ 0001 \\ +1 \\ \hline 0010 \end{array}$$

Department of Electronics & Communication

www.cambridge.edu.in

Floating-point representation:

Floating-point numbers are so called as the decimal or binary point floats over the base depending on the exponent value. It consists two components.

- Exponent
- Mantissa

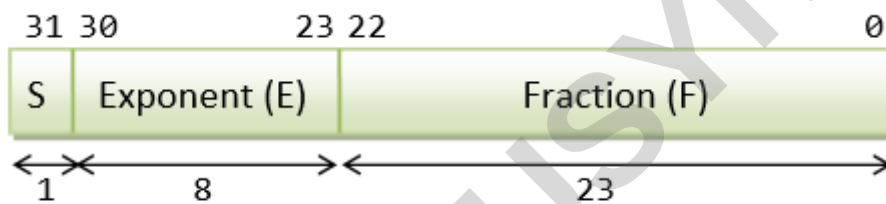
Table – 1 Precision Representation

Precision	Base	Sign	Exponent	Significand
Single precision	2	1	8	23+1
Double precision	2	1	11	52+1

IEEE-754 32-bit Single-Precision Floating-Point Numbers

In 32-bit single-precision floating-point representation:

- The most significant bit is the *sign bit* (S), with 0 for positive numbers and 1 for negative numbers.
- The following 8 bits represent *exponent* (E).
- The remaining 23 bits represents *fraction* (F).



32-bit Single-Precision Floating-point Number

Example 1: Convert $(3.5)_{10}$ to IEEE 32 bit floating point format

Solution:

1. Convert the given decimal number into its equivalent binary.
 $0.5 \times 2 = 1.0 \quad 1$
 $0.0 \times 2 = 0 \quad 0$ [not required to multiply with 0]
 $(3.5)_{10} = (11.1)_2$
2. Normalize the number [$1.M \times 2^{E'}$ format is used]
 1.11×2^1 (here $E' = 1$, the power of 2)
3. M = Mantissa = $110\ 0000\ 0000\ 0000\ 0000\ 0000$ [extra bits as mantissa is of 23 bits]
4. Exponent $E = E' + 127$
 $= 1 + 127 = (128)_{10} = (1000\ 0000)_2$
5. Sign bit $S = 0 \Rightarrow$ positive number
6. $0\ 1000\ 0000\ 110\ 0000\ 0000\ 0000\ 0000\ 0000$ sign, exponent and mantissa

Department of Electronics & Communication

www.cambridge.edu.in

Example 1: Suppose that IEEE-754 32-bit floating-point representation pattern is

$0\ 1000\ 0000\ 110\ 0000\ 0000\ 0000\ 0000\ 0000$.

obtain its normalized form.

Sign bit $S = 0 \Rightarrow$ positive number

$E = 1000\ 0000_2 = 128D$

Use the format $1.M$ and convert binary to decimal.

Fraction is $1.11B$ (with an implicit leading 1)

$$= 1 + 1 \times 2^{-1} + 1 \times 2^{-2} = 1.75D$$

Since the E is > 127 hence use the formula $E' = E - 127$

The number is $+1.75 \times 2^{(128-127)} = +3.5D$

Example 2: Suppose that IEEE-754 32-bit floating-point representation pattern is $1\ 01111110\ 100\ 0000\ 0000\ 0000\ 0000\ 0000$. obtain its normalized form.

Sign bit $S = 1 \Rightarrow$ negative number

$E = 0111\ 1110_2 = 126D$ (in normalized form)

Fraction is $1.1B$ (with an implicit leading 1) $= 1 + 2^{-1} = 1.5D$

The number is $-1.5 \times 2^{(126-127)} = -0.75D$

Department of Electronics & Communication

www.cambridge.edu.in

Binary, Signed-Integer Representations

B $b_3b_2b_1b_0$	Values represented		
	Sign and magnitude	1's complement	2's complement
0 1 1 1	+ 7	+ 7	+ 7
0 1 1 0	+ 6	+ 6	+ 6
0 1 0 1	+ 5	+ 5	+ 5
0 1 0 0	+ 4	+ 4	+ 4
0 0 1 1	+ 3	+ 3	+ 3
0 0 1 0	+ 2	+ 2	+ 2
0 0 0 1	+ 1	+ 1	+ 1
0 0 0 0	+ 0	+ 0	+ 0
1 0 0 0	- 0	- 7	- 8
1 0 0 1	- 1	- 6	- 7
1 0 1 0	- 2	- 5	- 6
1 0 1 1	- 3	- 4	- 5
1 1 0 0	- 4	- 3	- 4
1 1 0 1	- 5	- 2	- 3
1 1 1 0	- 6	- 1	- 2
1 1 1 1	- 7	- 0	- 1

Figure 2.1. Binary, signed-integer representations.

2's-Complement Add and Subtract Operations

(a)	$\begin{array}{r} 0010 \\ + 0011 \\ \hline 0101 \end{array}$	$\begin{array}{l} (+2) \\ (+3) \\ \hline (+5) \end{array}$	(b)	$\begin{array}{r} 0100 \\ + 1010 \\ \hline 1110 \end{array}$	$\begin{array}{l} (+4) \\ (-6) \\ \hline (-2) \end{array}$
(c)	$\begin{array}{r} 1011 \\ + 1110 \\ \hline 1001 \end{array}$	$\begin{array}{l} (-5) \\ (-2) \\ \hline (-7) \end{array}$	(d)	$\begin{array}{r} 0111 \\ + 1101 \\ \hline 0100 \end{array}$	$\begin{array}{l} (+7) \\ (-3) \\ \hline (+4) \end{array}$
(e)	$\begin{array}{r} 1101 \\ - 1001 \\ \hline 0100 \end{array}$	$\begin{array}{l} (-3) \\ (-7) \\ \hline (+4) \end{array}$			
(f)	$\begin{array}{r} 0010 \\ - 0100 \\ \hline 1110 \end{array}$	$\begin{array}{l} (+2) \\ (+4) \\ \hline (-2) \end{array}$			
(g)	$\begin{array}{r} 0110 \\ - 0011 \\ \hline 0101 \end{array}$	$\begin{array}{l} (+6) \\ (+3) \\ \hline (+3) \end{array}$			
(h)	$\begin{array}{r} 1001 \\ - 1011 \\ \hline 1110 \end{array}$	$\begin{array}{l} (-7) \\ (-5) \\ \hline (-2) \end{array}$			
(i)	$\begin{array}{r} 1001 \\ - 0001 \\ \hline 1000 \end{array}$	$\begin{array}{l} (-7) \\ (+1) \\ \hline (-8) \end{array}$			
(j)	$\begin{array}{r} 0010 \\ - 1101 \\ \hline 0101 \end{array}$	$\begin{array}{l} (+2) \\ (-3) \\ \hline (+5) \end{array}$			

Figure 2.4. 2's-complement Add and Subtract operations.

Overflow Conditions

5	0111 0101	-7	1000 1001
3	0011	-2	1100
-8	1000	7	1011
Overflow		Overflow	
5	0000 0101	-3	1111 1101
2	0010	-5	1011
7	0111	-8	1100
No overflow		No overflow	

Overflow when carry-in to the high-order bit does not equal carry out

Overflow occurs in following conditions:

1. when adding two numbers that have the same sign.
2. the carry out signal from the sign bit position is not a sufficient indicator of overflow when adding signed numbers.

A simple way to detect overflow is to examine the signs of the two summands X and Y and the sign of the result. When both operands X and Y have the same sign, an overflow occurs when the sign of S is not the same as the signs of X and Y.



Sign Extension Example

Decimal	Hexa decimal	Binary (8 bit)	Sign Extended Binary (16 bit)
8	8	0000 1000	0000 0000 0000 1000
161	A1	1010 0001	1111 1111 1010 0001
13	0D	0000 1101	0000 0000 0000 1101
236	EC	1110 1100	1111 1111 1110 1100

MEMORY-LOCATIONS & ADDRESSES

- **Memory** consists of many millions of storage cells (flip-flops).
- Each cell can store a bit of information i.e. 0 or 1 (Figure 2.1).
- Each group of n bits is referred to as a **word** of information, and n is called the **word length**.
- The word length can vary from 8 to 64 bits.
- A unit of 8 bits is called a **byte**.
- Accessing the memory to store or retrieve a single item of information (word/byte) requires distinct addresses for each item location. (It is customary to use numbers from 0 through 2^k-1 as the addresses of successive-locations in the memory).
- If $2^k = \text{no. of addressable locations}$;
then 2^k addresses constitute the address-space of the computer.
For example, a 24-bit address generates an address-space of 2^{24} locations (16 MB).

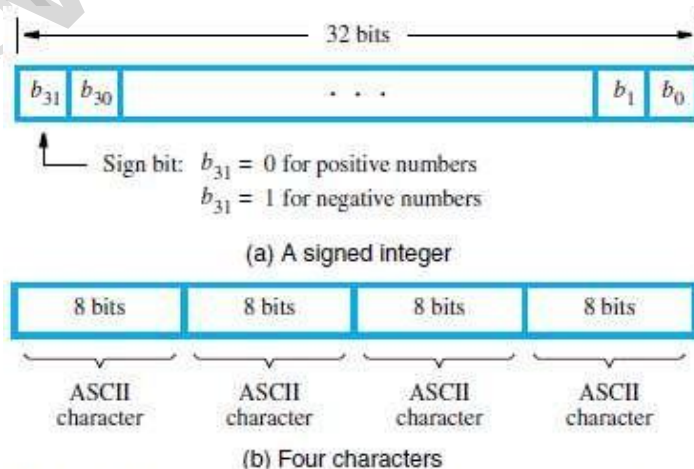
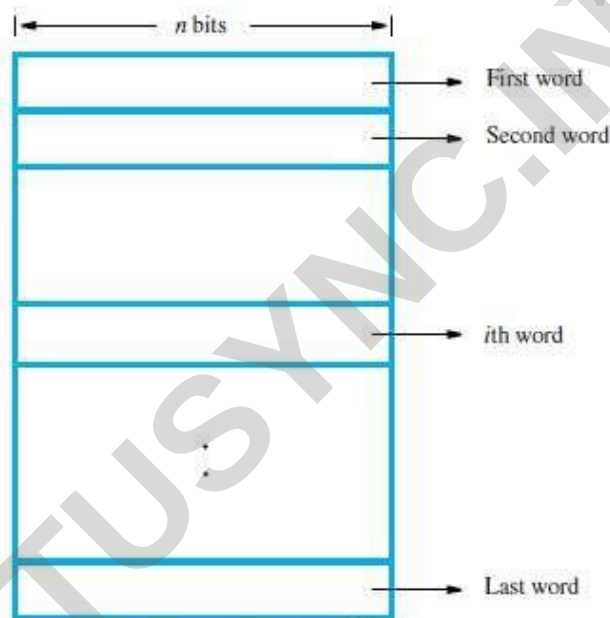


Figure 2.2 Examples of encoded information in a 32-bit word.

COMPUTER ORGANIZATION

BYTE-ADDRESSABILITY

- In byte-addressable memory, successive addresses refer to successive byte locations in the memory.
- Byte locations have addresses 0, 1, 2,
- If the word-length is 32 bits, successive words are located at addresses 0, 4, 8, . . with each word having 4 bytes.

BIG-ENDIAN & LITTLE-ENDIAN ASSIGNMENTS

- There are two ways in which byte-addresses are arranged (Figure 2.3).
 - 1) **Big-Endian:** Lower byte-addresses are used for the more significant bytes of the word.
 - 2) **Little-Endian:** Lower byte-addresses are used for the less significant bytes of the word
- In both cases, byte-addresses 0, 4, 8, are taken as the addresses of successive words in the memory.

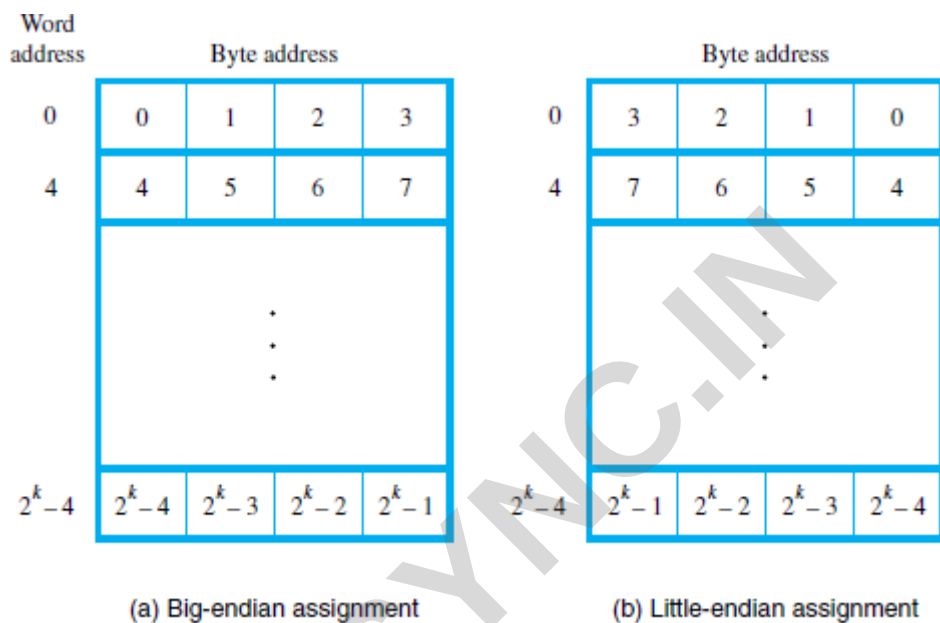


Figure 2.3 Byte and word addressing.

- Consider a 32-bit integer (in hex): 0x12345678 which consists of 4 bytes: 12, 34, 56, and 78.
 - Hence this integer will occupy 4 bytes in memory.
 - Assume, we store it at memory address starting 1000.
 - On little-endian, memory will look like

Address	Value
1000	78
1001	56
1002	34
1003	12

- On big-endian, memory will look like

Address	Value
1000	12
1001	34
1002	56
1003	78

WORD ALIGNMENT

- Words are said to be **Aligned** in memory if they begin at a byte-address that is a multiple of the number of bytes in a word.
- For example,
 - If the word length is 16(2 bytes), aligned words begin at byte-addresses 0, 2, 4,
 - If the word length is 64(2 bytes), aligned words begin at byte-addresses 0, 8, 16,
- Words are said to have **Unaligned Addresses**, if they begin at an arbitrary byte-address.

COMPUTER ORGANIZATION

ACCESSING NUMBERS, CHARACTERS & CHARACTERS STRINGS

- A number usually occupies one word. It can be accessed in the memory by specifying its word address. Similarly, individual characters can be accessed by their byte-address.
- There are two ways to indicate the length of the string:
 - 1) A special control character with the meaning "end of string" can be used as the last character in the string.
 - 2) A separate memory word location or register can contain a number indicating the length of the string in bytes.

MEMORY OPERATIONS

- Two memory operations are:
 - 1) Load (Read/Fetch) &
 - 2) Store (Write).
- The **Load** operation transfers a copy of the contents of a specific memory-location to the processor. The memory contents remain unchanged.
- Steps for Load operation:
 - 1) Processor sends the address of the desired location to the memory.
 - 2) Processor issues „read' signal to memory to fetch the data.
 - 3) Memory reads the data stored at that address.
 - 4) Memory sends the read data to the processor.
- The **Store** operation transfers the information from the register to the specified memory-location. This will destroy the original contents of that memory-location.
- Steps for Store operation are:
 - 1) Processor sends the address of the memory-location where it wants to store data.
 - 2) Processor issues „write' signal to memory to store the data.
 - 3) Content of register(MDR) is written into the specified memory-location.

INSTRUCTIONS & INSTRUCTION SEQUENCING

- A computer must have instructions capable of performing 4 types of operations:
 - 1) Data transfers between the memory and the registers (MOV, PUSH, POP, XCHG).
 - 2) Arithmetic and logic operations on data (ADD, SUB, MUL, DIV, AND, OR, NOT).
 - 3) Program sequencing and control (CALL, RET, LOOP, INT).
 - 4) I/O transfers (IN, OUT).

COMPUTER ORGANIZATION

REGISTER TRANSFER NOTATION (RTN)

- The possible locations in which transfer of information occurs are: 1) Memory-location 2) Processor register & 3) Registers in I/O device.

Location	Hardware Binary Address	Example	Description
Memory	LOC, PLACE, NUM	$R1 \leftarrow [LOC]$	Contents of memory-location LOC are transferred into register R1.
Processor	R0, R1, R2	$[R3] \leftarrow [R1] + [R2]$	Add the contents of register R1 & R2 and places their sum into R3.
I/O Registers	DATAIN, DATAOUT	$R1 \leftarrow DATAIN$	Contents of I/O register DATAIN are transferred into register R1.

ASSEMBLY LANGUAGE NOTATION

- To represent machine instructions and programs, assembly language format is used.

Assembly Language Format	Description
Move LOC, R1	Transfer data from memory-location LOC to register R1. The contents of LOC are unchanged by the execution of this instruction, but the old contents of register R1 are overwritten.
Add R1, R2, R3	Add the contents of registers R1 and R2, and places their sum into register R3.

BASIC INSTRUCTION TYPES

Instruction Type	Syntax	Example	Description	Instructions for Operation $C \leftarrow [A] + [B]$
Three Address	Opcode Source1, Source2, Destination	Add A, B, C	Add the contents of memory-locations A & B. Then, place the result into location C.	
Two Address	Opcode Source, Destination	Add A, B	Add the contents of memory-locations A & B. Then, place the result into location B, replacing the original contents of this location. Operand B is both a source and a destination.	Move B, C Add A, C
One Address	Opcode Source/Destination	Load A	Copy contents of memory-location A into accumulator.	Load A Add B Store C
		Add B	Add contents of memory-location B to contents of accumulator register & place sum back into accumulator.	
		Store C	Copy the contents of the accumulator into location C.	
Zero Address	Opcode [no Source/Destination]	Push	Locations of all operands are defined implicitly. The operands are stored in a pushdown stack.	Not possible

- Access to data in the registers is much faster than to data stored in memory-locations.

- Let R_i represent a general-purpose register. The instructions:

Load A, R_i

Store R_i, A

Add A, R_i

are generalizations of the Load, Store and Add Instructions for the single-accumulator case, in which register R_i performs the function of the accumulator.

- In processors, where arithmetic operations are allowed only on operands that are in registers, the task $C \leftarrow [A] + [B]$ can be performed by the instruction sequence:

Move A, R_i

Move B, R_j

Add R_i, R_j

Move R_j, C

INSTRUCTION EXECUTION & STRAIGHT LINE SEQUENCING

- The program is executed as follows:
 - 1) Initially, the address of the first instruction is loaded into PC (Figure 2.8).
 - 2) Then, the processor control circuits use the information in the PC to fetch and execute instructions, one at a time, in the order of increasing addresses. This is called *Straight-Line sequencing*.
 - 3) During the execution of each instruction, PC is incremented by 4 to point to next instruction.
- There are 2 phases for Instruction Execution:
 - 1) **Fetch Phase:** The instruction is fetched from the memory-location and placed in the IR.
 - 2) **Execute Phase:** The contents of IR is examined to determine which operation is to be performed. The specified-operation is then performed by the processor.

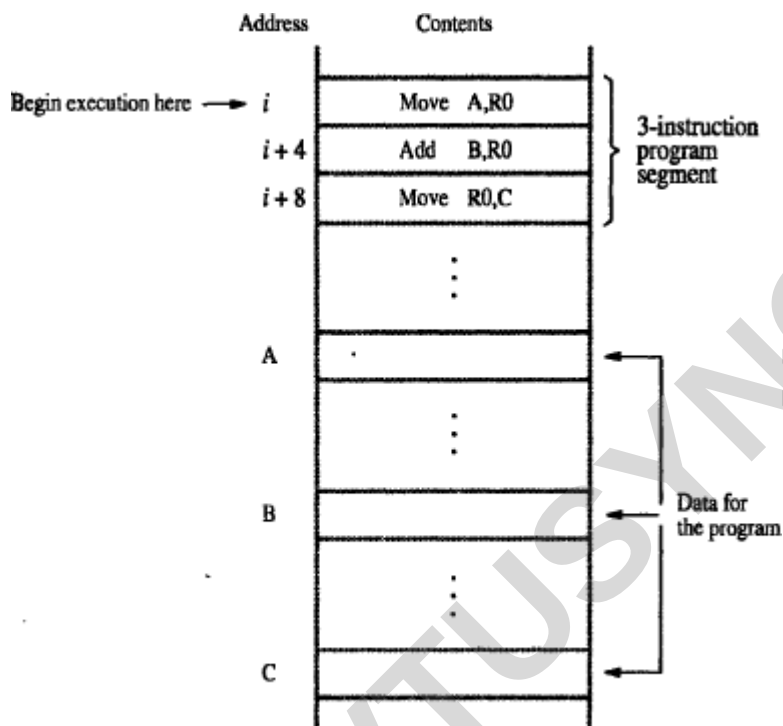


Figure 2.8 A program for $C \leftarrow [A] + [B]$.

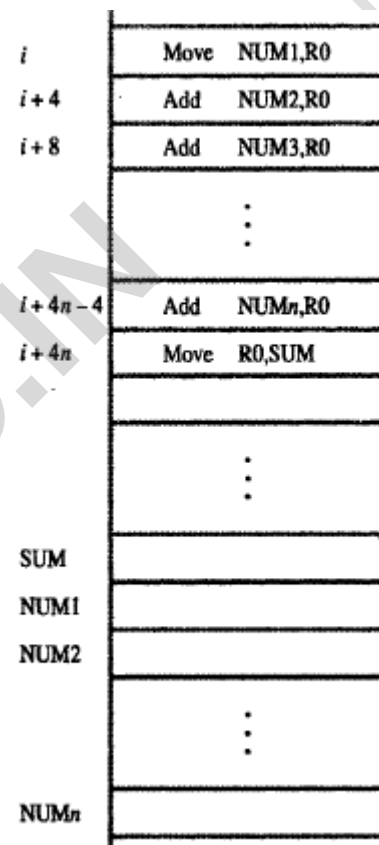


Figure 2.9 A straight-line program for adding n numbers.

Program Explanation

- Consider the program for adding a list of n numbers (Figure 2.9).
- The Address of the memory-locations containing the n numbers are symbolically given as NUM1, NUM2.....NUM n .
- Separate Add instruction is used to add each number to the contents of register R0.
- After all the numbers have been added, the result is placed in memory-location SUM.

BRANCHING

- Consider the task of adding a list of „n' numbers (Figure 2.10).
- Number of entries in the list „n' is stored in memory-location **N**.
- Register **R1** is used as a counter to determine the number of times the loop is executed.
- Content-location N is loaded into register R1 at the beginning of the program.
- The **Loop** is a straight line sequence of instructions executed as many times as needed. The loop starts at location LOOP and ends at the instruction Branch>0.
- During each pass,
 - address of the next list entry is determined and
 - that entry is fetched and added to R0.
- The instruction *Decrement R1* reduces the contents of R1 by 1 each time through the loop.
- Then **Branch Instruction** loads a new value into the program counter. As a result, the processor fetches and executes the instruction at this new address called the **Branch Target**.
- A **Conditional Branch Instruction** causes a branch only if a specified condition is satisfied. If the condition is not satisfied, the PC is incremented in the normal way, and the next instruction in sequential address order is fetched and executed.

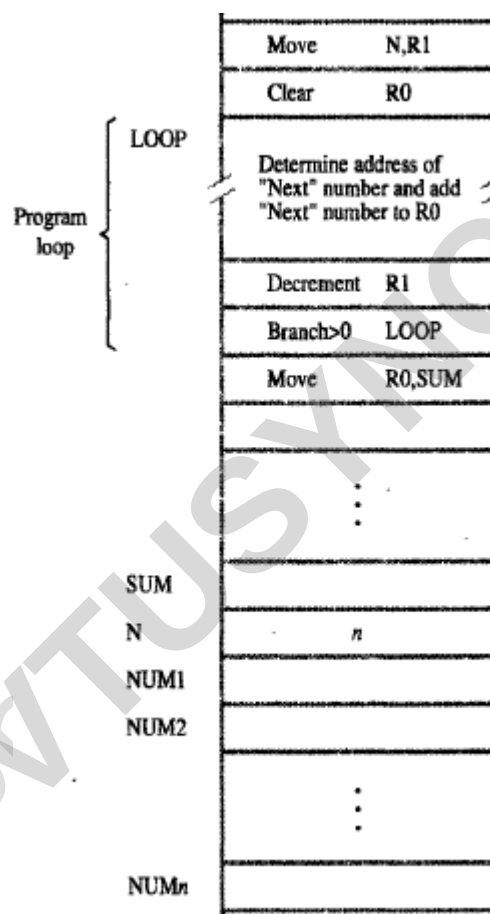


Figure 2.10 Using a loop to add n numbers.

CONDITION CODES

- The processor keeps track of information about the results of various operations. This is accomplished by recording the required information in individual bits, called **Condition Code Flags**.
- These flags are grouped together in a special processor-register called the condition code register (or statue register).
- Four commonly used flags are:
 - 1) **N** (negative) set to 1 if the result is negative, otherwise cleared to 0.
 - 2) **Z** (zero) set to 1 if the result is 0; otherwise, cleared to 0.
 - 3) **V** (overflow) set to 1 if arithmetic overflow occurs; otherwise, cleared to 0.
 - 4) **C** (carry) set to 1 if a carry-out results from the operation; otherwise cleared to 0.